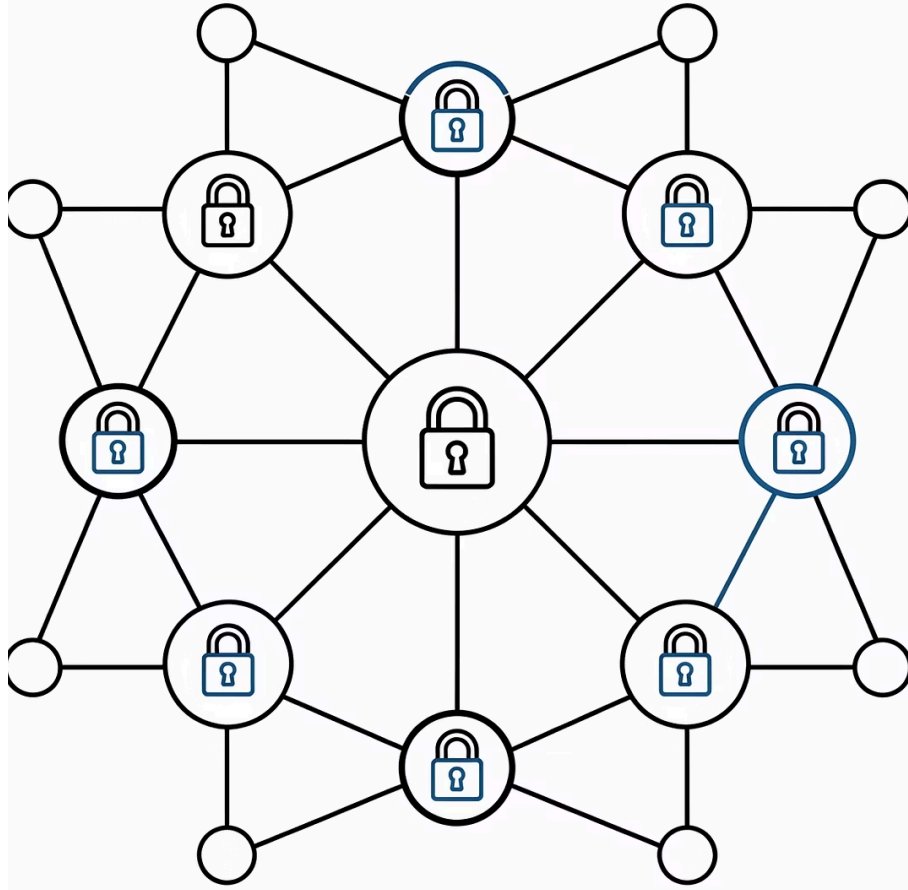
Vulnerability Management and the AI Crossroads

How AI Is Reshaping What We Find, What We Fix, and How Fast We Have to Move

Jerry Gamblin (@jgamblin)
Founder, RogoLabs
ITESO | June 3, 2026



Before We Begin

The Question That Drives This Session

> Several years focused on one question: *how do we make vulnerability data actually usable?* That is what "CVE Decaf" is about: strip the noise, keep the signal. Same protection, none of the alert-fatigue jitters from chasing 40,000 CVEs a year.

The Problem

AI finds bugs faster than humans can fix them. It also outpaces the NVD's ability to catalog them.

The Frame

Actionable data quality over raw volume: every claim anchored to a number, tool, or source

The Goal

Frameworks you can apply on day one of a security role, not theory

By the End of Today You Will Be Able To...

01

Explain CVE Forecasting

Why CVE forecasting shifted from a time-series to a capability-triggered problem

02

Apply the Exploitability Overlay

Use KEV + EPSS to reduce a raw CVE feed of 40,000 entries to an actionable target list

03

Describe the MTTR Race

Articulate the race dynamics and the organizational limits that no AI tool can overcome

04

Identify Ephemeral Risk

Explain why AI-generated code creates an attack surface that CVE scanners cannot see

05

Recognize Human Constraints

Identify the human capacity limits that define and bottleneck the entire CVD ecosystem

What We'll Cover Today

Section 1 - 09:00–09:45

The AI-driven surge in vulnerability discovery, and why it is mostly noise.

Section 2 - 09:45–10:50

Filtering signal from noise using KEV and EPSS exploitability overlays

Break - 10:50–11:05

15 minutes - live API exercise at `api.first.org`

Section 3 - 11:05–11:55

The MTTR race: AI on offense and defense, with real organizational limits

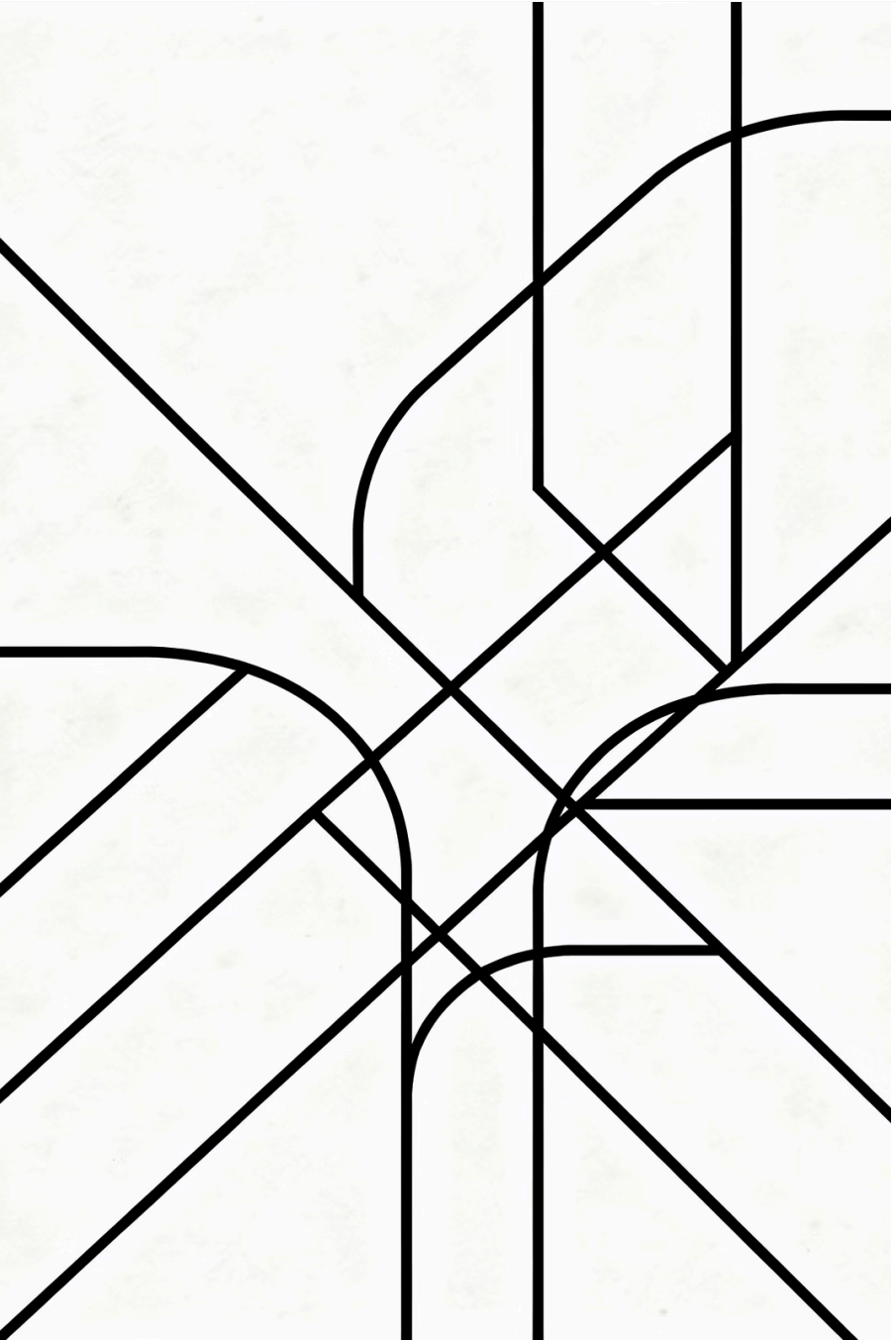
Section 4 - 11:55–12:40

Ephemeral software: the attack surface CVEs can't capture

The Central Question

If AI can find bugs faster than humans can fix them, and faster than NVD can even catalog them, **what does "secure" even mean?**

We'll return to this question at the end of the session with a precise, usable answer.



Opening Question

If AI Can Find Bugs Faster Than Humans Can Fix Them...

...and faster than the NVD can even catalog them...

...what does "secure" even mean?

We will return to this question in the final section with a specific, actionable answer. Hold it in mind as we go.

SECTION 1

From Automation to Autonomous AI

09:00 – 09:45 · The AI-driven surge in vulnerability discovery, the structural break in CVE forecasting, and why raw volume is the wrong signal

---

Where We Started

Rule-based automation: scan, flag, triage by hand

Where We Are

AI discovers vulnerabilities end-to-end and drafts its own fixes

Why It Matters

Not a degree shift: a categorical one. The operating model has changed.

We've Always Automated Security

Automation Era: 2010s

Rule-based scanners: Nessus, Qualys, OpenVAS

SIEM correlation rules written and maintained by humans

Humans triage findings → humans approve updates → humans deploy

The bottleneck was always the human in the middle

Agentic AI Era: 2025–2026

AI discovers vulnerabilities end-to-end. No human initiates the scan.

Agents draft fixes and open pull requests automatically

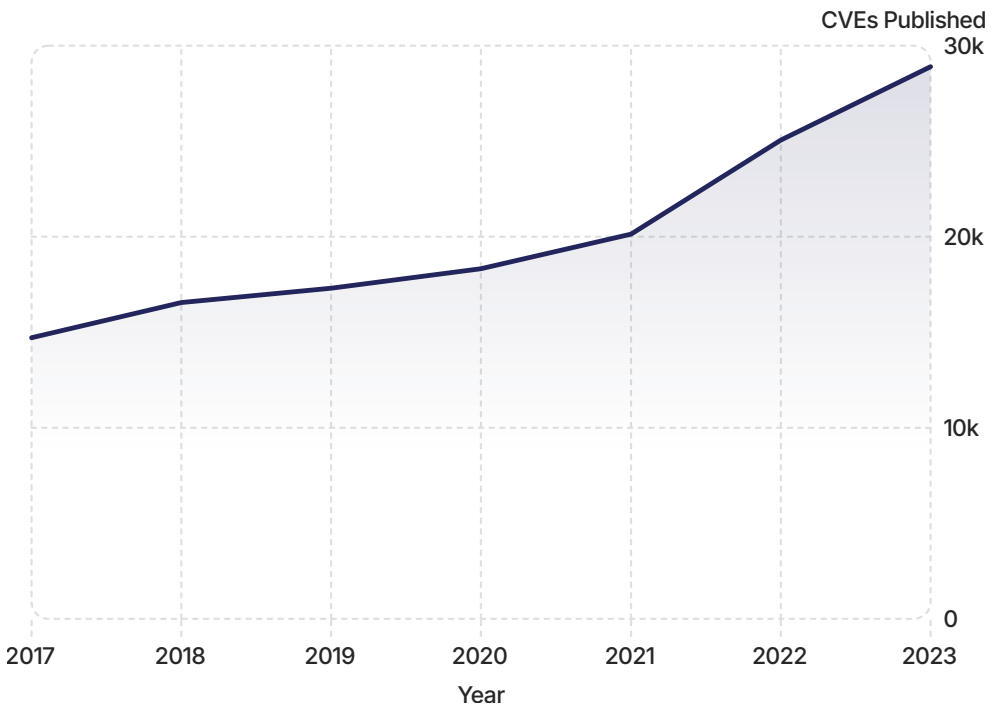
Humans approve and systems deploy. The loop is dramatically shorter.

The bottleneck shifts to organizational processes, not individual triage

⚠ The shift is not one of degree: it is one of kind. A faster scanner is the same model. An agent that discovers, drafts, and deploys is a different model entirely.

CVE Growth Was Once Predictable

CVE (Common Vulnerabilities and Exposures) is a unique identifier for a publicly disclosed software vulnerability.



The Old Forecasting Model

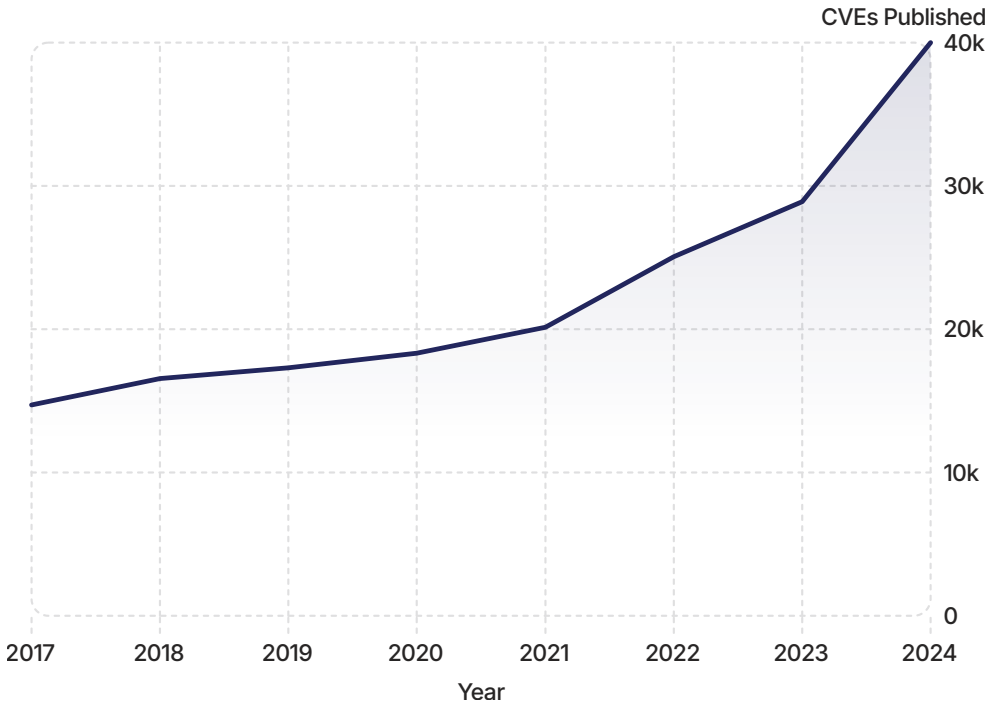
Annual CVE disclosures grew at roughly **12% CAGR from 2017-2023** (NVD published totals). Time-series models worked: last year × 1.12 was good enough for headcount and tooling budgets.

Security teams could plan accordingly. The curve was smooth, human-bounded, and legible to finance teams.

Source: NVD/MITRE published annual totals. Data ref: `01_cve_discovery_analysis.ipynb`

Who forecasts CVE volume and why: government agencies tracking critical infrastructure risk, security vendors publishing annual threat reports, and enterprise security teams budgeting headcount and tooling. Forecasting accuracy directly determines whether those budgets are adequate.

Then the Trendline Broke



The Capability-Triggered Model

Old model: growth bounded by the number of human security researchers. Smooth, predictable, linear.

New model: growth bounded by AI capability, with discontinuous jumps whenever a better model ships. Each major AI capability release is a potential step-change in disclosure volume.

⚠ This is the presenter's framing - "capability-triggered model" - not an established industry term. It is descriptively accurate, not a citation.

Forecasting audiences: government agencies tracking infrastructure risk, security vendors publishing trend reports, and enterprise teams budgeting headcount and tooling. When the model breaks, all three get their planning assumptions wrong simultaneously.

The Infrastructure Cracked Before the Surge

The 2024 NVD Crisis

In **May 2024**, NIST announced the suspension of CVE enrichment for most new entries in the National Vulnerability Database. Tens of thousands of CVE IDs were assigned but unanalyzed. They were invisible to downstream scanners relying on NVD enrichment for CVSS scores and CPE mappings.

Root cause: insufficient human staffing at a single critical infrastructure node.

The Consensus Engine

The **Consensus Engine** is the open-source community response to the NVD enrichment backlog: a single developer's tooling that continuously cross-checks the global vulnerability infrastructure by comparing CVE feeds, NVD enrichment status, EPSS scores, and KEV entries for gaps and discrepancies.

Cost: $0. When a government agency is the single point of failure for global security data, individual practitioners with the right tooling can run independent audits. This is accountability, not replacement.

AI discovery volume makes independent data-quality auditing **more** important, not less.
The ecosystem cannot rely on a single enrichment node.

The sustainable fix is not more NIST staffing. It is holding CNAs (CVE Numbering Authorities: the organizations authorized to assign CVE IDs) accountable for record quality at the time of creation. Pushing enrichment responsibility upstream to the issuer reduces the bottleneck at the source rather than concentrating it at a single government agency.

Real AI-Assisted Vulnerability Research

These are **documented, shipping tools**: not future projections or marketing claims.

Google OSS-Fuzz + LLM Analysis

Automated fuzzing with AI-assisted crash triage. Has found thousands of real bugs in critical open-source software including OpenSSL, libpng, and SQLite.

GitHub Copilot Autofix

AI-suggested security fixes inline in code review: shown directly in the PR diff. Reduces fix latency from days to minutes for common vulnerability classes.

Microsoft Security Copilot

AI assistant for threat intelligence, alert triage, and vulnerability context. Integrated with Sentinel and Defender. Generally available, in active enterprise deployment.

Commercial AI Fuzzers

Mayhem (ForAllSecure) and Jazzer are production tools for automated vulnerability discovery: used in regulated industries including aerospace and financial services.

~38% More CVEs in a Single Year

28,900

CVEs in 2023

NVD published total, a record at the time.

~40K

CVE IDs Assigned (MITRE)

~38% single-year jump, the largest in program history.

400+

Authorized CNAs

Up from ~100 in 2017; more issuers mean more CVE numbers

Three structural drivers: AI-assisted bug hunting, CNA expansion (100 → 400+ authorized issuers since 2017), and retroactive disclosure with NVD backlog clearing. Volume is up for tooling and structural reasons. It does not mean software suddenly became much worse.

Discussion: "Is this good news or bad news?" - Both. More coverage is better. More noise is worse. The question is what you do with the signal.

You Can No Longer Budget Off the Raw CVE Feed

Volume Is a Function of AI Capability Releases

Raw CVE count is no longer bounded by human researcher capacity. Each major model release is a potential step-change. Year-over-year comparisons are misleading without normalizing for AI tooling adoption rates.

The Signal You Care About Is Decoupled From the Noise

Total disclosures ≠ exploitable risk. The subset that can actually hurt your organization grows at a fundamentally different rate than the raw feed. You need a filter: not a bigger team to chase every CVE.

What Is the Right Signal?

Exploitability-filtered prioritization: only the CVEs confirmed as actively exploited or predicted to be exploited imminently. That is what the next section covers.

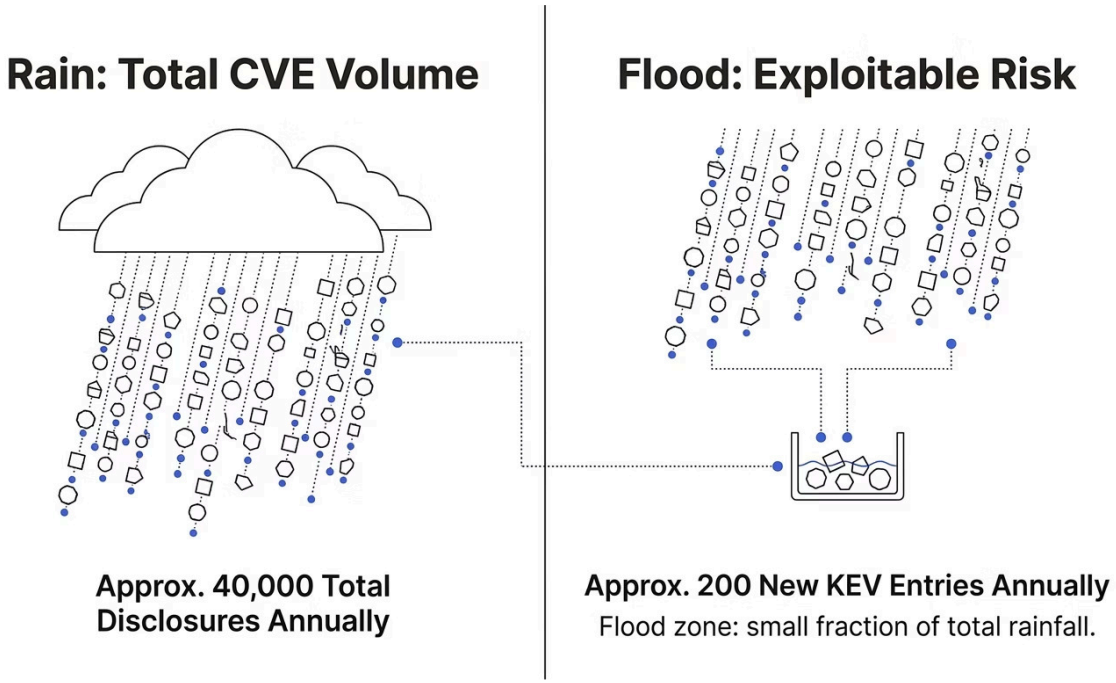
SECTION 2

The Exploitability Overlay

09:45 – 10:50 · Rain vs. Floods: turning **40,000 CVEs** into an actionable target list

---

KEV (Known Exploited Vulnerabilities: CISA's catalog of CVEs confirmed as actively exploited in the wild) and **EPSS (Exploit Prediction Scoring System: an ML model that scores the probability a CVE will be exploited within 30 days)** are the filters used to separate actionable risk from the full stream of disclosures.



We've Always Found More Than We Can Fix

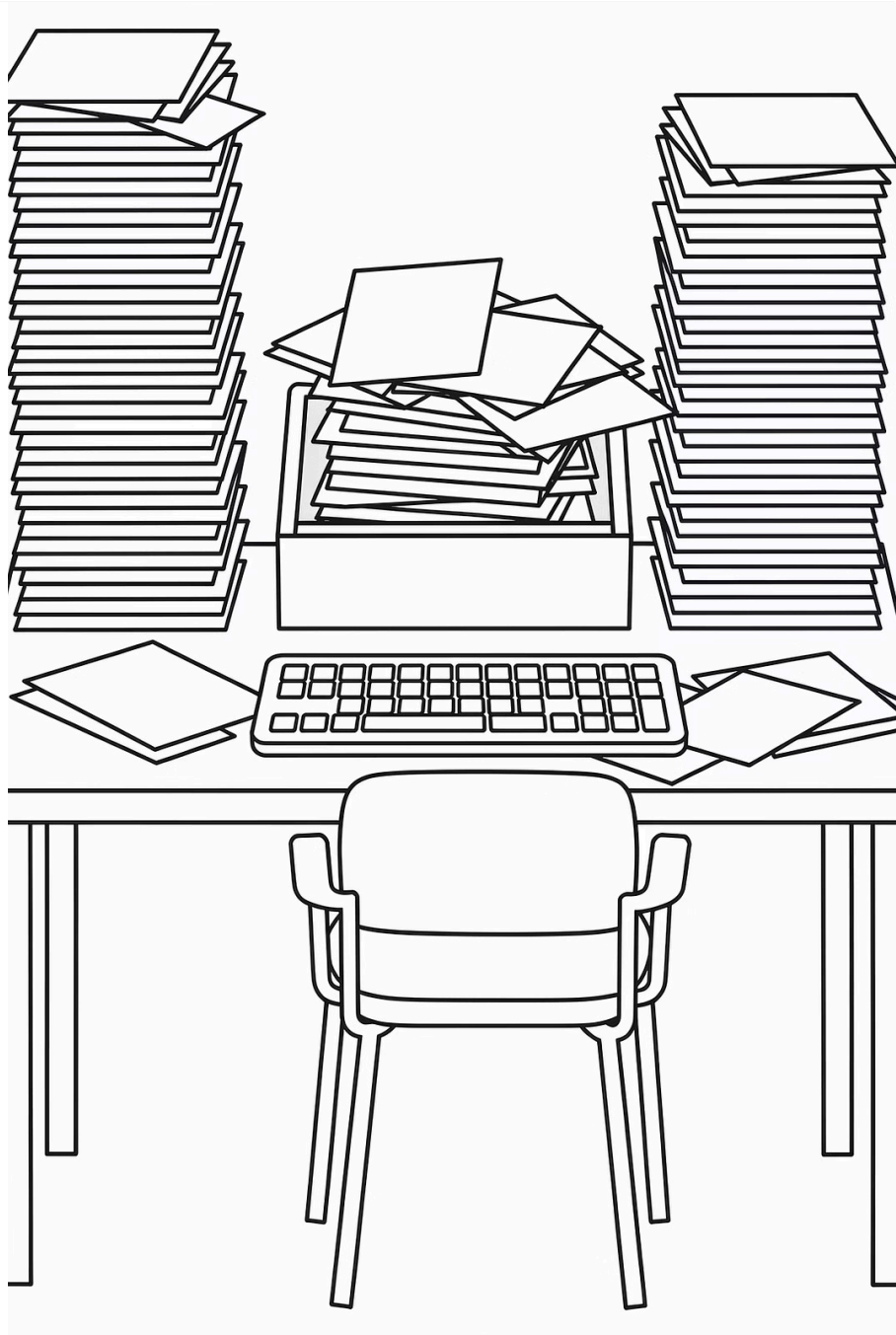
The Real Bottleneck

Discovery has *never* been the constraint in security operations. Human capacity - to verify, coordinate, prioritize, test, and deploy - is always the limit. AI expanding discovery widens the gap; it amplifies an existing problem, not a new one.

The Math That Matters

⚠ Updating burden = CVEs × affected assets × environments. **50 KEV entries × 10,000 servers = 500,000 remediation instances.** The number of CVEs is the least important variable.

> The constraint is not what we know. The constraint is what we can act on.



Not All Rain Becomes Floods

🌧️ Rain

Total CVE volume: every disclosed vulnerability, exploitable or not. **~40,000 CVEs in 2024.** Includes theoretical issues, legacy software nobody runs, unmaintained systems with no network exposure, and edge cases that require chained exploitation.

🌊 Flood

Actionable exploitable risk: the subset that can hurt you right now. **~200 new KEV entries per year.** Confirmed in-the-wild exploitation. These are the vulnerabilities with working exploit code already running in attacks against real targets.

Heavy rainfall does not necessarily cause flooding. Terrain, drainage, and elevation matter. Your asset inventory, network exposure, and update velocity are your drainage system.

KEV is backward-looking. It records confirmed exploitation with a lag. Watch for that lag shrinking as AI accelerates exploit development. A lagging indicator with a shorter lag is still a lagging indicator.

Three Reasons the Rain Is Getting Heavier

1

AI-Assisted Bug Hunting

Automated pipelines finding issues at scale: OSS-Fuzz, Copilot Autofix, commercial fuzzers. Bugs that would have taken a researcher a week now surface in an overnight run.

2

CNA Expansion

From ~100 to 400+ authorized CVE issuers since 2017. More organizations can assign CVE numbers, so more things get CVE numbers: including classes of issues that previously went undocumented.

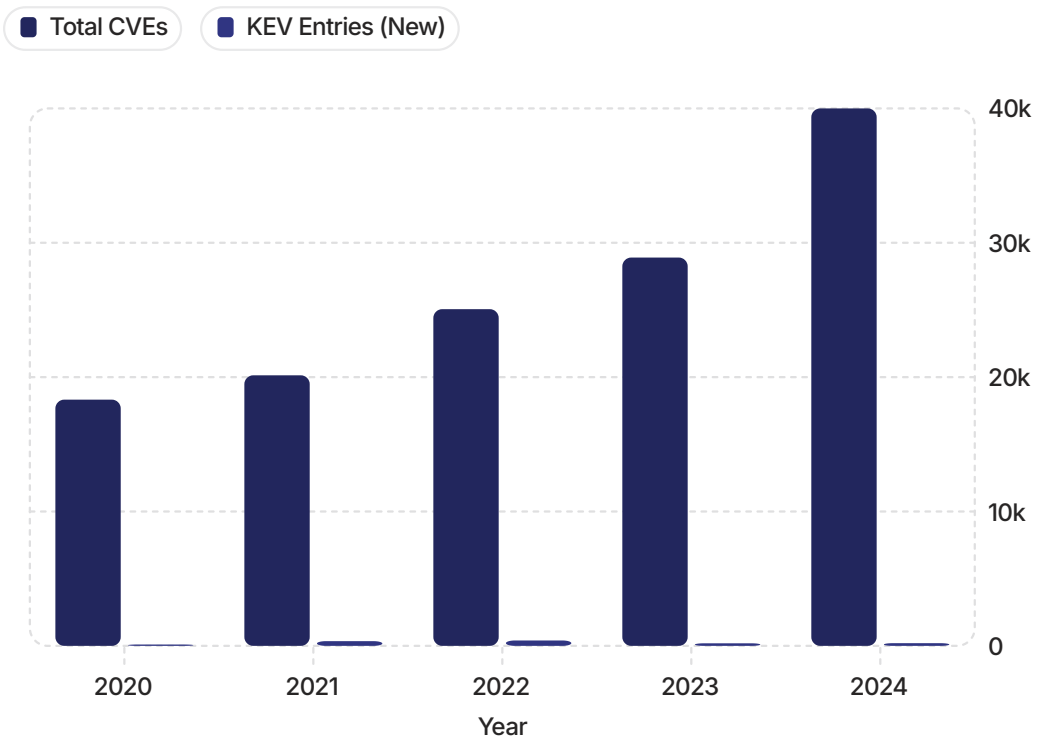
3

Retroactive Disclosure

Old issues being formally documented and cataloged as the CVD ecosystem matures. The NVD backlog clearing also contributed to apparent volume spikes in some years.

Key takeaway: volume is up for structural and tooling reasons. It does not mean software is suddenly catastrophically worse. It means the documentation system is catching up and AI is lowering the cost of finding bugs.

The Actionable Subset Grows Much More Slowly



The Two Filters

CISA KEV catalog: ~1,200 cumulative entries of confirmed in-the-wild exploits as of early 2026. ~150–300 new entries per year in recent years.

EPSS > 10%: a practitioner-convention threshold that filters ~85–90% of CVE volume, keeping only the subset with meaningful exploitation probability.

Noise-to-signal ratio: ~40,000 new CVEs in 2024 against ~200 new KEV entries: **200:1**.

Data ref:

`02_exploitability_overlay.ipynb`

The Ratio That Changes Everything

~40,000

CVE IDs Assigned (MITRE)

Total disclosures in 2024

~200

New KEV Entries

Confirmed in-the-wild exploits in 2024

200:1: That is the noise-to-signal ratio. Apply the filter.

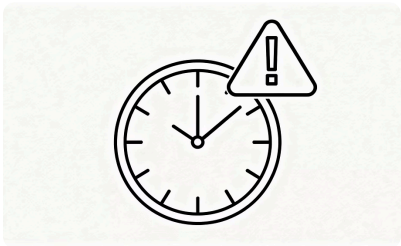
KEV: The Update List That's Not Optional

What It Is

Maintained by CISA. Contains only CVEs with confirmed, active exploitation in the wild. ~1,200 total entries as of early 2026.

Who Must Comply

Federal agencies are *legally required* to deploy the updated software for KEV entries under Binding Operational Directive 22-01. Best practice for everyone else. Live catalog: `cisa.gov/known-exploited-vulnerabilities-catalog`



The Critical Caveat

KEV is a **lagging indicator**. By the time a CVE appears in KEV, active exploitation has already been confirmed. It is a floor, not a ceiling: your minimum acceptable target.

EPSS: The Probability Score That Changes Prioritization

What EPSS Is

Exploit Prediction Scoring System (first.org/epss) is an ML model trained on exploitation telemetry from honeypots, threat feeds, and incident reports. Outputs a probability (0–100%) that a CVE will be observed in exploitation activity within **30 days**. Updated daily; responds to new threat intelligence.

EPSS > 10% is a widely-used practitioner convention, not an official standard. Tune to your own risk appetite.

CVSS vs. EPSS: A Critical Distinction

CVSS

How bad a vulnerability could be. Severity is based on technical characteristics.

EPSS

How likely it is to actually appear in real attacks. It measures probability, not severity.

⚠ High CVSS ≠ High EPSS. Many CVSS 9.x CVEs have EPSS < 1%. Many CVSS 5.x CVEs are actively exploited. Never prioritize on CVSS alone.

Calibrate to Your Risk Appetite

The Four-Tier Software Update Deployment Framework

Tier	Filter	~Annual CVE Count (2024)	Best For
Minimum Viable	KEV only	~200 new entries/year	Resource-constrained teams; absolute floor
Recommended	KEV + EPSS > 10%	~2,000–4,000/year	Most organizations; best ROI on deploying the updated software effort
Comprehensive	EPSS > 1%	~8,000–15,000/year	High-value targets; regulated industries
Exhaustive	All CVEs	~40,000 (2024); ~82K projected 2026	Nobody. This is not a program: it's a treadmill.

⊗ These are raw CVE counts. Your actual remediation burden = CVE count × affected assets × environments. A 10-server shop and a 100,000-server enterprise have very different workloads from the same CVE count.

More Rain, Slower-Growing Flood Lines

AI discovery is increasing CVE volume: structural and will continue

Capability-triggered growth means each major model release is a potential step-change. Plan for discontinuous increases, not smooth 12% annual growth.

The exploitable subset is not growing at the same rate

Exploitability filters (KEV + EPSS) remain effective at cutting noise. The 200:1 ratio is actionable, but only if you apply the filter.

Watch KEV's lag: it is shrinking

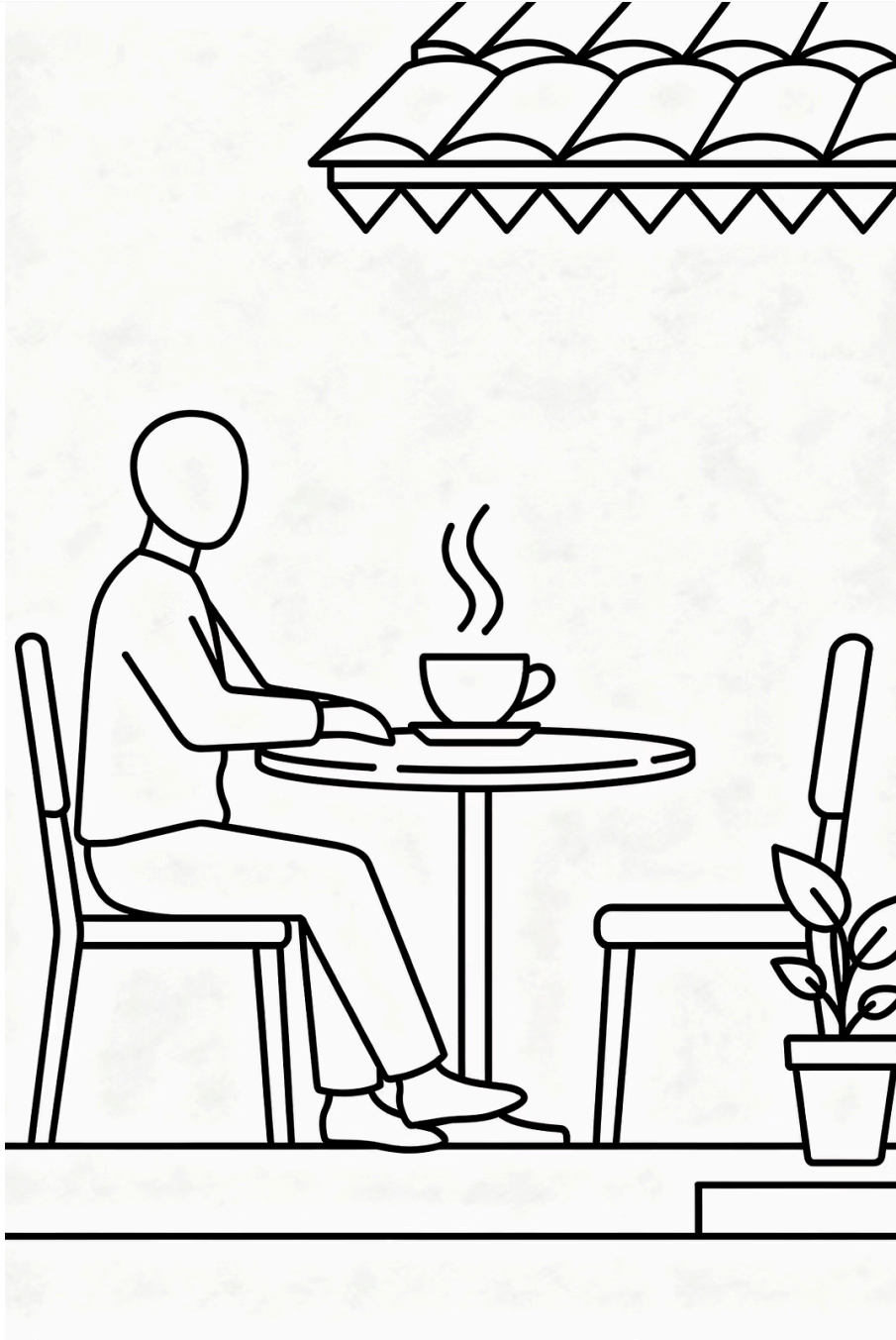
AI-accelerated exploit development compresses the time between public disclosure and confirmed exploitation. A lagging indicator with a shorter lag requires faster response cycles.

Up next: What happens when AI accelerates the exploit side of the equation, and what defensive tools actually exist to fight back?

15-Minute Break

Back at 11:05

Try this while you're up: Open a browser and go to `api.first.org/data/v1/epss?cve=CVE-2021-44228`: that's Log4Shell. What's the current EPSS score? Is it what you'd expect for a 3-year-old, extremely well-known vulnerability? Why or why not? We'll discuss when we return.



SECTION 3

Defensive AI and the MTTR Race

11:05 – 11:55 · Poachers Turning Gamekeepers: AI on offense, AI on defense, and the organizational limits that neither side can automate away

---

Offense

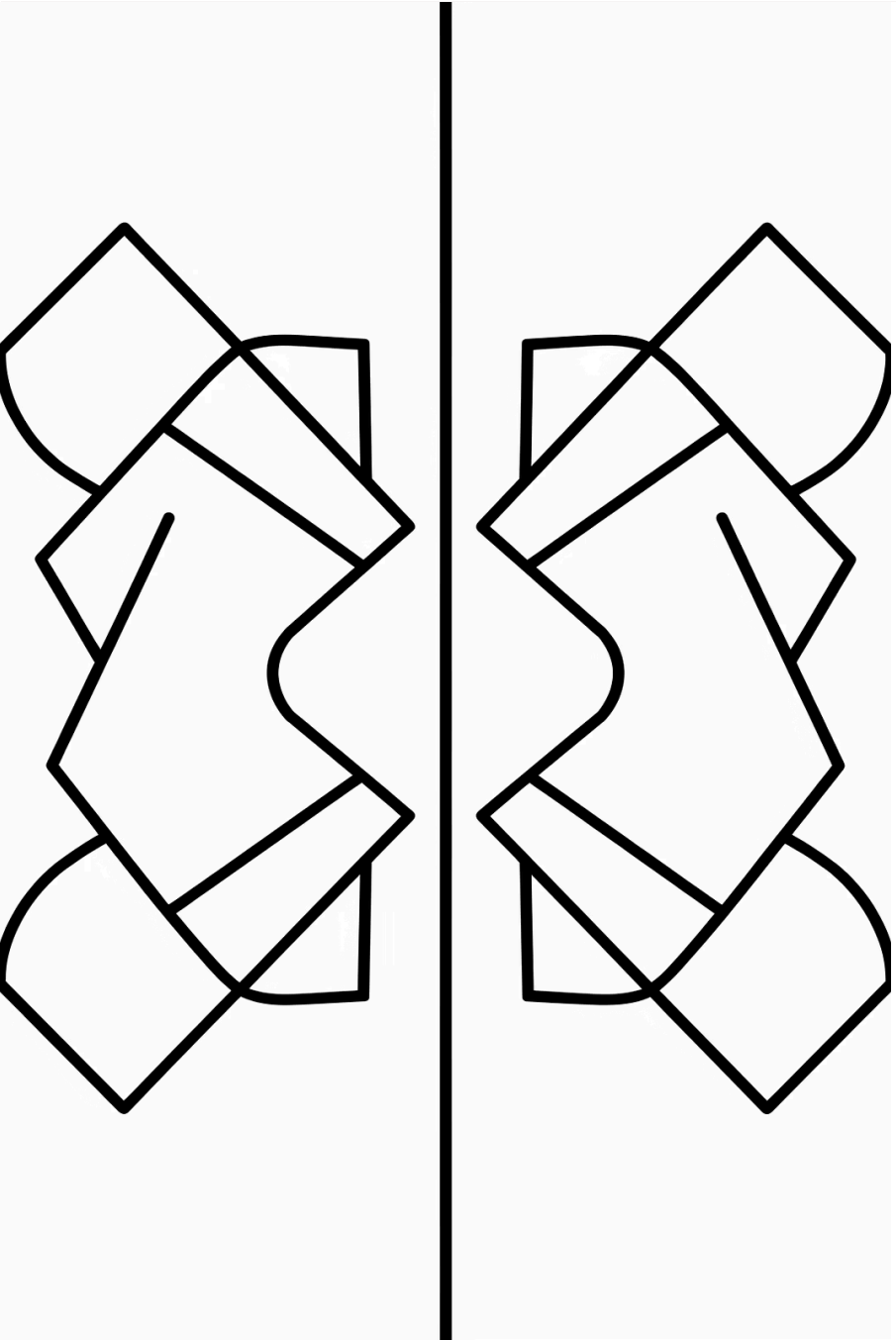
AI generating working PoCs in hours for known vulnerability classes

Defense

Shipping tools: Security Copilot, Charlotte AI, Copilot Autofix - real, in deployment

The Floor

Change boards, vendor schedules, and regulatory windows that AI cannot compress



The Same Capability Works Both Ways

Understanding a code path well enough to find a bug is nearly identical to understanding it well enough to fix it. Every major offensive AI advance becomes a defensive capability within months: the asymmetry does not persist.

1. **Fuzzing**

   Developed for offense: became the standard for defensive pre-release testing

2. **Static Analysis**

   Code auditing tools: SAST integrated into every major CI/CD pipeline

3. **AI Exploit Gen**

   LLMs generating PoC code: AI fix generation and Autofix entering production

The Defensive Stack Is Catching Up

These are **shipping and in active enterprise use**: not research prototypes or future roadmap items.

Microsoft Security Copilot

AI assistant for threat intelligence, incident investigation, and alert triage. Integrated with Sentinel and Defender. Generally available.

CrowdStrike Charlotte AI

Conversational AI for threat hunting and alert explanation inside the Falcon platform. Reduces analyst time-to-understand on complex detections.

GitHub Copilot Autofix

AI-suggested security fixes inline in the development workflow. Surfaces fixes at the moment a vulnerability is introduced: not weeks later in a scan report.

Snyk / Semgrep AI

AI-assisted vulnerability triage and fix suggestion in CI/CD pipelines. Reduces false-positive fatigue by ranking findings by exploitability and fix confidence.

Mean Time to Remediate: The Metric That Matters

MTTR Defined

Time from CVE disclosure (or internal detection) to **deployed updated software in production**. Not "updated version available." Not "PR merged." Production.

Industry median for critical vulnerabilities: 60–80 days pre-AI era

Source: Tenable "Life of a Vulnerability" report; Edgescan annual statistics

The Five Components of MTTR

Time to Know

Vulnerability is disclosed and reaches your team's awareness

Time to Prioritize

Triaged against KEV/EPSS and assigned to an owner

Time to Release an Updated Version

Fix drafted, reviewed, tested: **AI can compress this step**

Time to Test

Regression and functional validation

Time to Deploy the Updated Software

Change approval, maintenance window, production push

⚠ AI can compress steps 2-4 for in-house code. Steps 1 and 5 are largely unchanged. Vendor-dependent releasing an updated version bypasses AI entirely.

AI Can't Fix What It Doesn't Own

The Structural Reality

Most enterprise attack surface is **third-party software**: Oracle, SAP, Cisco, VMware, Microsoft, and hundreds of commercial applications. For vendor software, you wait for the vendor's release schedule: an AI drafting a fix is irrelevant.

Emergency updates for legacy ERP systems can take **6–18 months** to validate due to dependency chains, certification requirements, and contractual restrictions on modification.

⊗ AI-accelerated MTTR is real for greenfield, in-house, cloud-native code. For the rest - which is *most* of an enterprise - the old timeline applies.

What AI Can Accelerate

- In-house application code
- Open-source dependencies you maintain
- Cloud-native microservices
- Infrastructure-as-code

What AI Cannot Touch

- Oracle, SAP, Cisco, VMware updates
- Vendor-signed firmware
- Legacy ERP systems
- Hardware with embedded software

Case Study: Patch in 4 Hours, Deployed in 47 Days

Setting: Mid-size regulated bank. Java web portal using a vulnerable Apache Commons component. CVE published Monday morning, EPSS 0.58, KEV-listed within 48 hours.

Hour 0: CVE Published

MITRE assigns ID, EPSS score: 0.58

Day 2: KEV Listed

CISA adds to Known Exploited Vulnerabilities catalog

Day 31: Staging Deploy

QA and regression testing in staging environment

Hour 4: Updated Version Available

Vendor releases fix; AI-assisted fix generation

Day 12: Update Approved

Change Advisory Board review and sign-off

Day 47: Production Deploy

Updated software finally live in production

> **The bottleneck was never the update. It was the process.**

AI-Accelerated Exploit Development

What Changed

Pre-AI: Working proof-of-concept for a published CVE took days to weeks. A skilled exploit developer was required. This gave defenders a meaningful window.

Post-AI: Working PoC can be generated within hours for well-understood vulnerability classes, including buffer overflows, injection, and deserialization. The window is compressing.

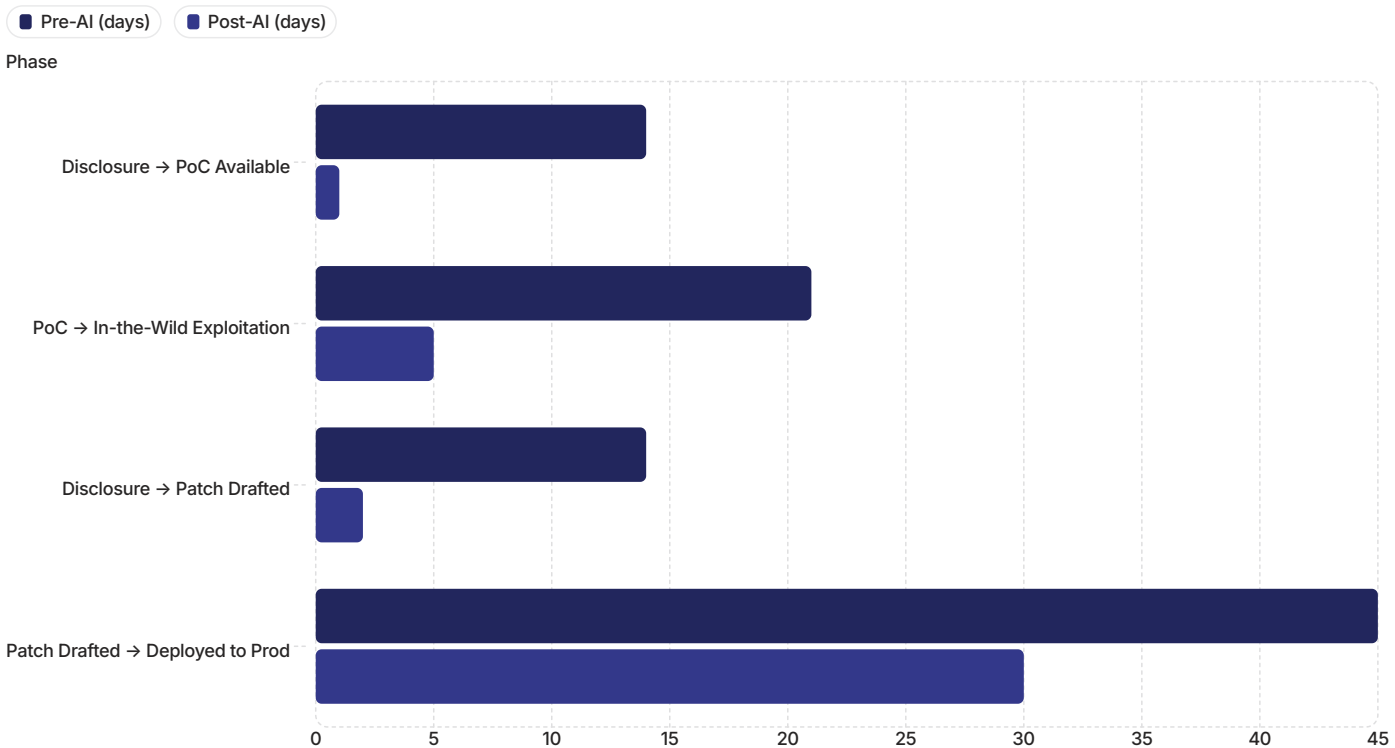
The Operational Impact

The old operational assumption that "we have a week to deploy the updated software for critical CVEs" no longer holds for high-profile vulnerabilities in well-understood software classes. For a Log4Shell-class event in 2026, assume hours, not days.

This is not theoretical. Documented cases of AI-assisted PoC development appearing within 24 hours of CVE publication exist for multiple 2024–2025 vulnerabilities.

What Matters Is the Delta

Window of Exposure: Pre-AI vs. Post-AI (In-House Code)



Both timelines compress under AI: but the exploit side compresses *faster* than the deployment side. Organizational deployment processes set a floor that technical tooling cannot move. The race is won or lost in process improvement, not tool selection. Data ref: `03_explotation_timeline.ipynb`. Timelines are illustrative medians for in-house code.

The Bottleneck AI Can't Fix

Change Approval Boards

Floor of 5–10 days for standard changes at most enterprises. Exists for legitimate reasons: someone must own every production change.

Regression Testing Requirements

Organizations with slow MTTR often lack the test coverage to deploy faster safely. Test debt is MTTR debt.

Vendor-Dependent Updating

Oracle, SAP, Cisco release an updated version on their schedule. Deploying the updated software without vendor approval can void support contracts.

Regulatory Freeze Windows

Banks and healthcare systems have periods where no production changes are allowed, including quarter-end and audit periods.

Discussion: "Which of these controls would you eliminate? Which would you keep if your name was on the change record?" Every delay in the 47-day case mapped to one of these categories. None was bureaucratic waste.

The MTTR Race Is Already Underway

Offense Is Already There

Exploit generation has crossed into the **hours timescale** for known vulnerability classes in well-understood software. The safe window has compressed. Organizations that assume a week still have a week should re-examine that assumption.

Defense Is Real and Deployed

Security Copilot, Charlotte AI, and Copilot Autofix are not prototypes. They are in active enterprise deployment today. The gap is narrowing, but it requires investment and integration to capture the benefit.

The Dangerous Middle

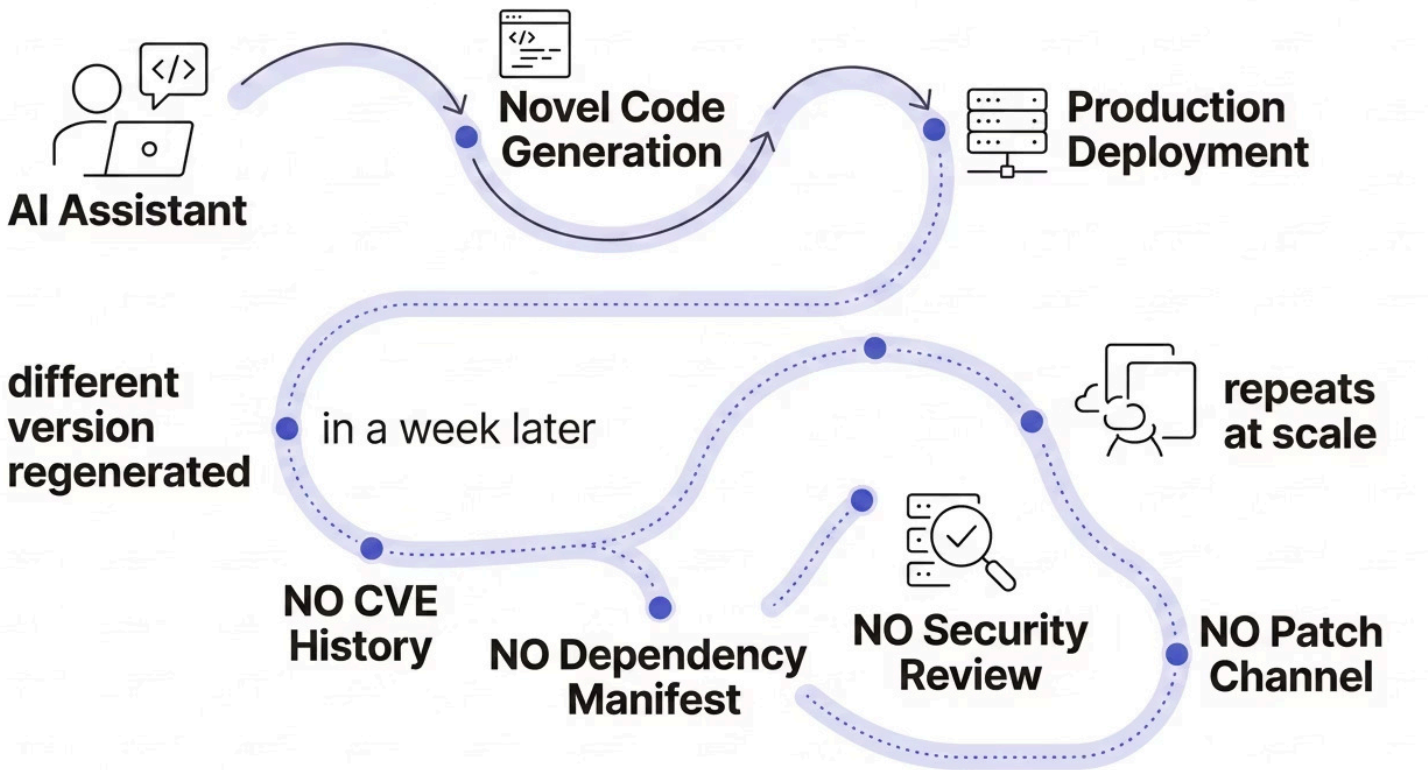
Organizations that *automate discovery* but cannot *accelerate deployment*, whether for process or vendor-dependency reasons, are in the worst position. They face more findings, the same time to remediate, and a larger visible exposure gap.

Up next: There is one attack surface we have not discussed. It is the one no CVE will ever cover.

SECTION 4

Ephemeral Software and Micro-Vulnerabilities

11:55 – 12:40 · The Attack Surface CVEs Cannot See. AI-generated code, bespoke vulnerabilities, and the tooling gap define the security research agenda for 2025 through 2030



Traditional VM Assumes a Stable Inventory

01

You Have a Software Asset Register

Every library, framework, and binary tracked. Version numbers known. Ownership assigned.

02

Libraries Have Known CVEs

Vulnerabilities are filed by CNAs, enriched by NVD, and indexed by your scanner against CPE data.

03

Vendors Issue Updated Versions

An updated version exists, has a CVE reference, and arrives through known channels: package manager, vendor portal, advisory email.

04

You Deploy the Updated Software

The entire system works because software changes slowly, is cataloged, and has institutional ownership.

⊗ AI assistants are breaking every one of these assumptions simultaneously.

The Shadow Registry

Traditional Software

01

Developer writes code

02

Code review

03

Dependency scan

04

CVE catalog match

05

Approved & deployed

Every component has a CVE history.

AI-Generated Code

01

Developer prompts AI assistant

02

Novel code generated

03

Reviewed for correctness only

04

Merged & deployed to production

No CVE catalog entry exists. No updated version will ever come.

⊗ **AI-generated code creates a shadow registry: bespoke vulnerabilities invisible to every scanner that relies on CVE matching.**

Not Like Log4j

Property	Log4j (Traditional)	AI-Generated Code Vuln
Scope	One library, millions of deployments	Unique per instance, bespoke
CVE Filed	CVE-2021-44228: immediately indexed	No CVE will ever be filed
Release Exists	Log4j 2.16.0: universal fix	No universal update; must fix per instance
Scanner Visibility	Every dependency scanner detects it	Invisible to NVD-based scanners
Update Channel	Distributed through existing package managers	No channel; requires manual audit
Persistence	Released an updated version globally within weeks for most deployments	May run for years before anyone audits it

⊗ The properties that made Log4j manageable: universality, a single CVE, a single update, are exactly the properties AI-generated vulnerabilities lack. The response playbook does not transfer.

This Is Already Showing Up in Research

Pearce et al. 2022: "Asleep at the Keyboard?"

~40% of Copilot-generated code in security-relevant scenarios contained vulnerabilities. Tested across multiple CWE categories including buffer overflows, injection, and use-after-free. Published: arxiv.org/abs/2108.09293

Perry et al. 2023

AI code assistants can lead developers to write *less secure* code even when developers believe they are writing securely. The confidence effect amplifies the risk. Published: arxiv.org/abs/2211.03622

2024 - 2025 Follow-Up Studies

Consistent rates of input validation failures, integer handling errors, and injection vulnerabilities across multiple models, languages, and security scenarios. The signal is robust across research groups.

⚠ These vulnerabilities pass syntactic code review because they *look correct*. The flaw is behavioral, not structural. A reviewer checking for clean code will miss them.

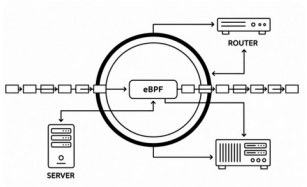
The Tooling Gap Is Real: Here's What We Have

Approach	What It Catches	Limitation
Static Analysis (bandit, semgrep, CodeQL)	Known bad patterns: injection, weak crypto, `shell=True`	Misses logic-level flaws; false positive fatigue reduces adoption
Mandatory Human Review on security-sensitive AI code	Anything a trained reviewer can spot	Doesn't scale; "security-sensitive" is difficult to define and enforce
Runtime Monitoring (Falco, eBPF, behavioral EDR)	Anomalous behavior in production	Reactive, not preventive; requires deep expertise to deploy universally
Restrict AI Codegen on Critical Paths	Prevents the problem at the source	Hard to enforce consistently at the developer tooling level

What to do today: Add static analysis to your CI/CD pipeline. Require human review on auth, data access, and external input handling. These are incomplete answers - but they are available now.

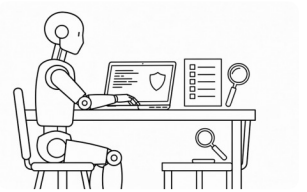
Where This Is Heading: Research Directions

None of these are production-ready universal answers. This is where the security engineering research agenda for 2025–2030 is focused, and where careers get built.



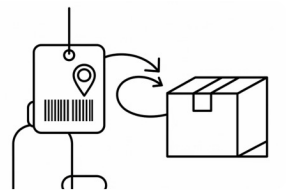
eBPF Runtime Enforcement

Cilium Tetragon moving beyond alerting toward blocking unexpected syscalls at the kernel level *before* they complete. Requires Linux kernel ≥ 5.8. Works cleanly only in cloud-native environments. Current limitation: deep expertise required.



AI-Agent Code Reviewers

Dedicated AI agents trained to review AI-generated code for security anti-patterns (Snyk DeepCode AI, CodeAnt AI, Semgrep AI). Open question: can AI reliably detect its own failure modes? Early results are mixed.



AI-Provenance Tagging

Annotating git commits and build artifacts with AI-generation metadata, enabling policy gates ("no AI-authored code in payment flow without AppSec sign-off"). No standard yet. Watch SLSA and SBOM extensions.

Scenario Question

You find AI-generated code in a production service during a security review.

No tests. No review record. `bandit` flags two medium-severity findings: one potential SQL injection, one `shell=True` subprocess call.

What do you do?

Immediate triage: disable, monitor, or accept risk? What is the escalation path? What constitutes evidence of exploitation?

Who do you involve?

Developer who generated it? Their manager? AppSec? Legal? Does the answer change if it's in a payment flow vs. an internal reporting tool?

What policy is missing?

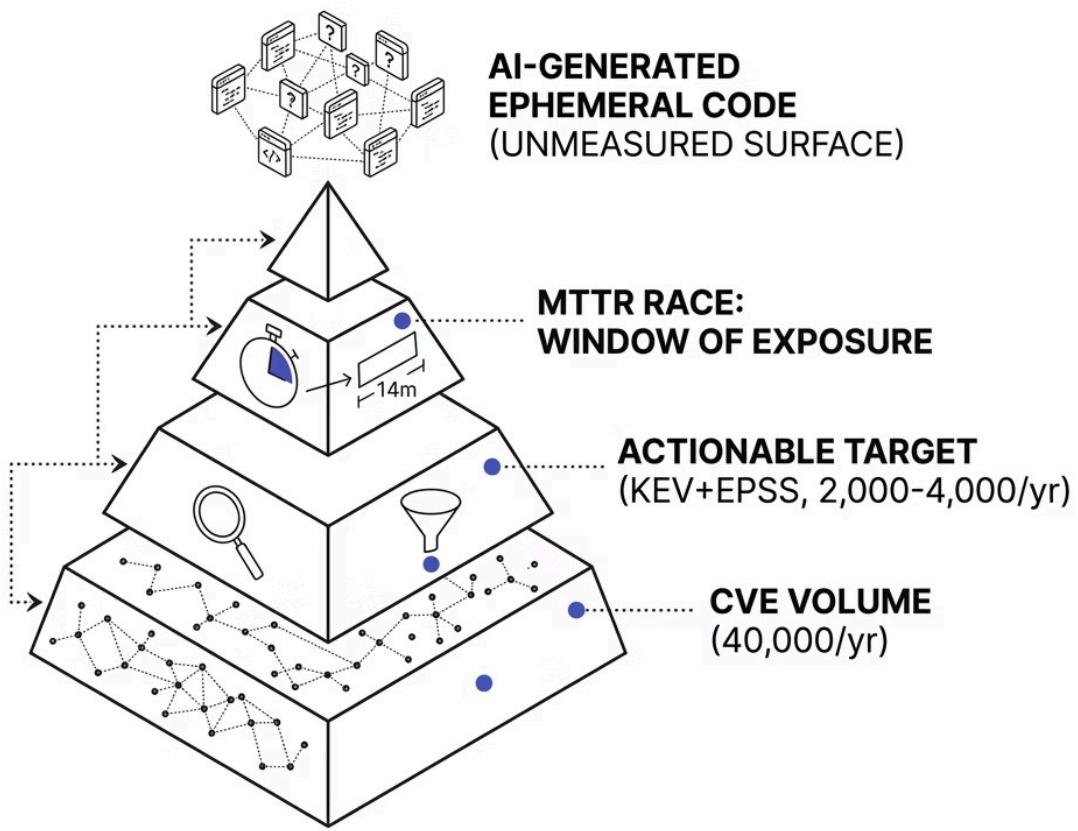
What governance would have prevented this? What is the minimum viable policy you could implement this week?

No single right answer. This scenario connects directly to your future roles as both developers and security engineers. The friction is real: get comfortable with it now.

SECTION 5

Conclusion

Returning to the Question We Started With



What Does "Secure" Mean in the AI Era?

> **Secure means maintaining a manageable, risk-calibrated Window of Exposure: not zero vulnerabilities.**

- **AI expands the attack surface faster than it shrinks MTTR: for now.**
- **Prioritize by exploitability (KEV + EPSS), not raw CVE volume.**
- **The binding constraint is human capacity, not AI capability.**
- **AI is not creating new vulnerability classes: it is revealing accumulated technical debt.**

  The same coding errors in the 2007 'Unforgivable Vulnerabilities' paper still dominate the 2025 CWE Top 25. Reducing software risk ultimately requires secure-by-design practices at the maker level, not just better deployment practices at the operator level.

The Binding Constraint Is Human Capacity

The CVD ecosystem runs on human decisions at every node: researcher, CNA, vendor, patch, and deployment.

The 2024 NVD Crisis: A Proof Point

A single understaffed human organization created a gap affecting the *entire global security ecosystem*. Tens of thousands of CVE IDs went unenriched for months, rendering them invisible to downstream scanners. This is not a failure of tooling. It is a failure of human organizational capacity at a critical infrastructure node.

The Correct Metric

More CVEs ≠ more risk. More unpatched exploitable vulnerabilities = more risk. These are different quantities. The entire discipline of vulnerability management is the work of keeping them separated.

What to Remember

01

AI Discovery Is Expanding CVE Volume Discontinuously

Capability-triggered, not linear. Mostly noise. Apply an exploitability overlay before any analysis or resourcing decision.

02

KEV + EPSS > 10% Is the Actionable Target

It grows much more slowly than total disclosures. The 200:1 noise-to-signal ratio is real and actionable: but only if you apply the filter. This is your minimum viable program.

03

MTTR Is the Critical Metric. Vendor Releasing an Updated Version Sets the Floor.

AI compresses technical steps for in-house code. Organizational processes, vendor schedules, and regulatory requirements set a floor that no tool can remove.

04

Ephemeral AI Code Is Invisible to CVE Scanners

Static analysis and mandatory review are the current best mitigations. Neither is complete. This is an open research problem and a career opportunity.

05

Human Capacity Is the Binding Constraint

To coordinate, decide, and deploy. This will not change. Build programs that respect this reality rather than assuming it away.

How to Resource Your Vulnerability Program

Resource to the size and growth rate of your software asset register: not to the raw CVE feed.

Intellectually Correct Framing

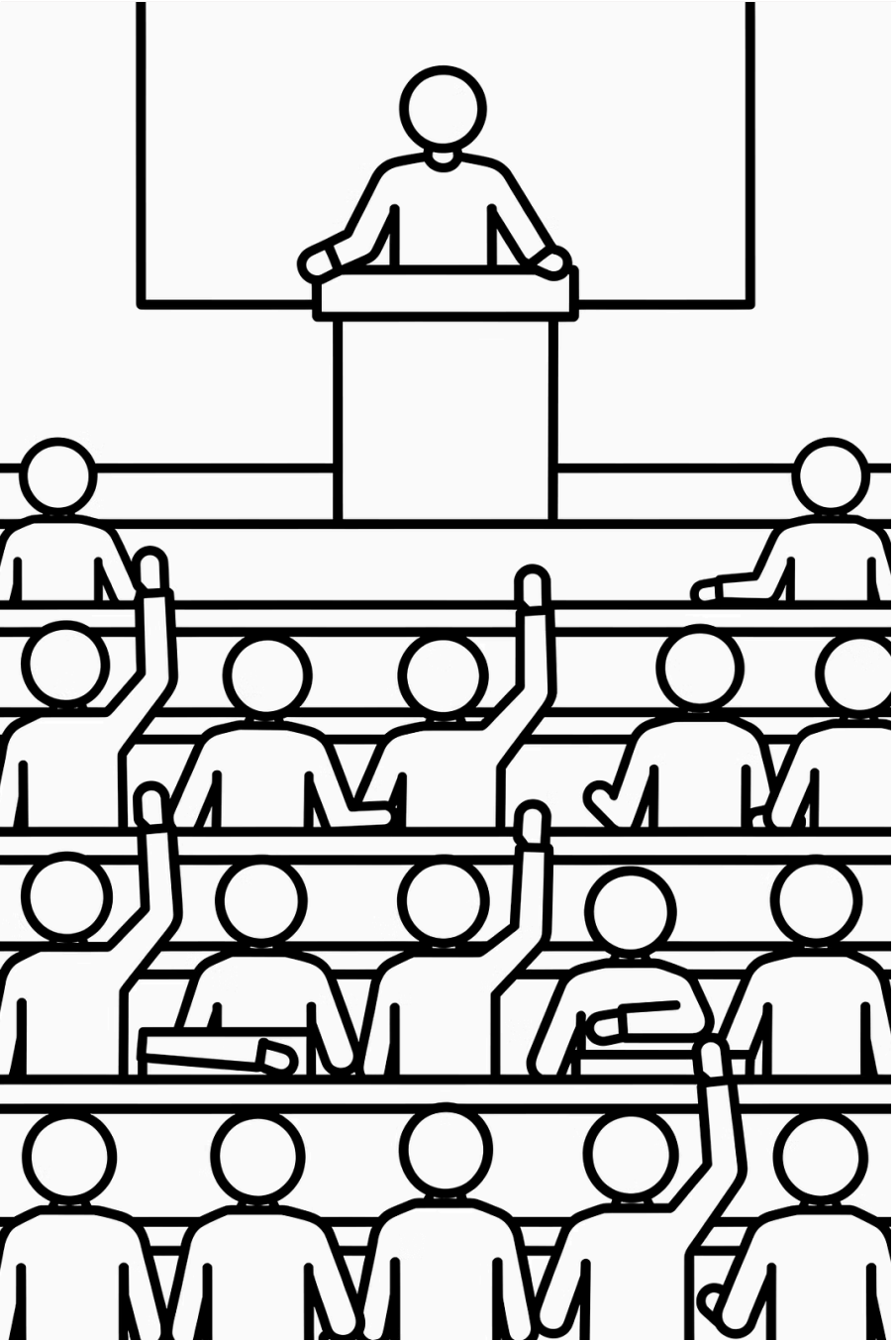
Asset register growth drives your true software update workload. A CVE that doesn't affect any asset you own costs you nothing. Size your team to the intersection of CVE volume and your actual asset footprint: not to the raw NVD feed.

Politically Effective Framing

What gets budget approved in the real world:

- Breach likelihood and historical incident cost
- Compliance exposure: SOC 2, PCI-DSS, HIPAA, ISO 27001
- Cyber insurance cost and coverage conditions
- Board-level risk appetite statements

Knowing both framings - and when to use each - is a professional skill. The technically correct argument loses the budget conversation more often than it should.



Open Floor

Suggested Discussion Starters

"What is your organization's software update SLA, and is it based on CVSS, KEV, EPSS, or something else? Do you know?"

"Has anyone encountered AI-generated code in a security review - intentionally or unintentionally? What did you find?"

"NIST suspended NVD enrichment in 2024. What does it mean for the ecosystem if a single agency is a critical bottleneck - and what should replace it?"

APPENDIX

Key Resources

CISA KEV Catalog

`cisa.gov/known-exploited-vulnerabilities-catalog`: The authoritative list of confirmed in-the-wild exploits. Subscribe to the feed. Deploy the updated software everything on it. This is your non-negotiable floor.

EPSS API + Docs

`first.org/epss` and `api.first.org/data/v1/epss?cve=CVE-XXXX-XXXXX`: Live probability scores, updated daily. Free API, no authentication required. Use it in your tooling today.

NVD API

`nvd.nist.gov/developers/vulnerabilities`: Programmatic access to the full CVE database with CVSS scores, CPE mappings, and enrichment status. Rate-limited; register for an API key.

Static Analysis Tools

Bandit (Python): `bandit.readthedocs.io` - Install with `pip install bandit`, run in 30 seconds. **Semgrep** (multi-language): `semgrep.dev` - 1,000+ rules, free tier available.

APPENDIX

Code Sample Index

All notebooks are designed to run against live APIs. Outputs will vary from slides - that is intentional. The data is real, not canned.

Notebook	What It Does
01_cve_discovery_analysis.ipynb	Real NVD API queries, CAGR analysis, structural break visualization. Reproduces the capability-triggered model chart with live data.
02_exploitability_overlay.ipynb	Live KEV download, EPSS fetch for your CVE list, CVSS vs. EPSS scatter plot, triage query function you can adapt for your environment.
03_exploitation_timeline.ipynb	Real exploitation lag data from KEV + NVD publication dates. SLA adequacy analysis - does your current MTTR target beat the median exploitation timeline?
04_ephemeral_software.ipynb	Live bandit scan of AI-generated code patterns. Visualizes the risk surface of common AI coding anti-patterns across CWE categories.

APPENDIX

Live Jupyter Notebooks: Download and Run

All code samples run against live APIs. Outputs will differ from the slides intentionally.

GitHub Repository

https://github.com/RogoLabs/ITESO-Lecture

Clone the repo and install requirements.txt. No API keys needed for KEV and EPSS.

Notebook	Covers
01_cve_discovery_analysis.ipynb	CVE volume trends, structural break, capability-triggered model
02_exploitability_overlay.ipynb	KEV download, EPSS fetch, CVSS vs. EPSS scatter
03_exploitation_timeline.ipynb	Exploitation lag data, SLA adequacy, MTTR benchmarking
04_ephemeral_software.ipynb	Bandit scan of AI-generated code, CWE visualization

APPENDIX

What This Lecture Didn't Cover

For students who ask what comes next — this is the full scope of a mature vulnerability management program. Today covered discovery, prioritization, MTTR, and ephemeral software. Everything below is a follow-on topic.

Asset Discovery

Finding all the assets before you can deploy the updated software. Shadow IT, cloud sprawl, and ephemeral compute make this harder than it sounds.

Ownership Chains

Who actually owns each update across teams? The RACI matrix for a critical update often spans 5–8 teams in a large enterprise.

Exception Management

Risk acceptance, compensating controls, and open exception tracking. Not everything gets deployed the updated software on schedule — how you manage the exceptions defines your actual risk posture.

SLA Definition

From what date does the MTTR clock start? Disclosure date? NVD publish date? Internal detection date? The answer materially changes your reported metrics.

VM Metrics and Board Reporting

What you measure is what gets resourced. Mean age of open critical findings, update compliance rate by tier, KEV closure rate — these are the numbers that move budgets.

APPENDIX

Glossary

CVE	Common Vulnerabilities and Exposures: unique identifier for a publicly disclosed vulnerability. Assigned by CNAs, indexed by MITRE and NVD.
CVSS	Common Vulnerability Scoring System: severity score (0–10) based on technical characteristics. Tells you how bad it *could* be, not how likely it is to be exploited.
EPSS	Exploit Prediction Scoring System: probability of observed exploitation activity in the next 30 days. ML-based, updated daily. `first.org/epss`
KEV	Known Exploited Vulnerabilities: CISA's catalog of confirmed in-the-wild exploits. The non-negotiable software update floor.
CNA	CVE Numbering Authority: organizations authorized to assign CVE IDs. Expanded from ~100 to 400+ since 2017.
MTTR	Mean Time to Remediate: average time from discovery to deployed updated software. The critical operational metric.
WoE	Window of Exposure: time between "exploit available" and "updated software deployed." This is what attackers are racing to exploit.
CVD	Coordinated Vulnerability Disclosure: the process by which researchers report findings to vendors before public disclosure.
SBOM	Software Bill of Materials: formal record of a software component's dependencies and provenance. Foundation for supply chain security.
Static Analysis	Automated inspection of source code for known vulnerability patterns without executing it. Tools: bandit, semgrep, CodeQL.